Polymorphic Encryption and Pseudonymisation

Release Management

Author: BvG, SR, JP

Version: 0.2

Date: 30-11-2017

Document Management

Change History

Date	Author	Version	Changes
31-10-2017	JP	0.1	First draft
30-11-2017	JP	0.2	Comments SR & BVG included

Distribution

Date	Version	To

Contents

Document Management 2
Change History 2
Distribution 2
Purpose of this document 3
Intended Audience 3
Release Management 3
Scope of PEP Release 3
Versions, Releases and hot-fixes 4
Release Progression 4
Development & Deployment 5
Authorizations 5
Procedures 6
Release procedure 6
Hot-Fix procedure 6

Purpose of this document

The purpose of this document is to describe the principles of change in the PEP software and infrastructure, and to provide procedures implementing these principles.

Intended Audience

The intended audience of this document is:

- The software development team
- Partners/Subcontractors involved in infrastructure management
- User representatives

Release Management

Is defined as the process of managing, planning, scheduling and controlling a software build through different stages and environments; including testing and deploying software releases.

Not within the scope of release management is:

- Management of development itself
- Functional evaluation of changes
- Impact evaluation of changes

Scope of PEP Release

The scope of the release management is the PEP system in a broad sense:

- The PEP server components, running on (vritualized) servers:
    - Keyserver
    - HSM
    - Authentication Server
    - Access Manager
    - Transcryptor
    - Storage Facility
    - Logger
- The PEP client
- The PEP infrastructure
    - Virtual environments for Build server, and all PEP components used for O and T are hosted by Digital Security, as is a physical DEV-HSM
    - Virtual servers hosting Key Server, Authentication Server and HSM's (A and P) environments are hosted by SURFnet
    - Virtual servers hosting the Transcryptor (A and P) are hosted by RU-ISC
    - Virtual servers hosting the Access Manager and Storage Facility (A and P) are hosted by RU-C&CZ.

    - Storage is hosted by RU-C&CZ for version 1 releases and by Google Infrastructure Services for Version 2 and 3.
- The PEP documentation
    - Risk Management
    - Release Management (this document)
    - …..

Versions, Releases and hot-fixes

Versions of PEP deliver sets of functionality related to the different stages in the PPP project (Parkinson op Maat/Personalized Parkinson's Project; https://www.parkinsonopmaat.nl/studie) . Currently there are three versions of PEP foreseen: Version 1 supporting the Data Collection phase, Version 2 supporting the Data Management phase and Version 3 supporting the Research Data Repository phase. Versioning is used to manage requirements and expectations from the users regarding the functional development of the PEP system.

Each **version** will contain a number of releases. A **release** contains a predefined set of functional and technical features that can be deployed to the production system. The rationale behind multiple releases is that functionality can be added in a gradual, organic way, where each release holds also a limited number of changes. In addition, technical improvements or quality improvements, transparent to the user, may be implemented in releases.

If a problem occurs in the production environment, posing a serious and acute threat to security, availability, integrity or confidentiality of the data, a **hot-fix** can be developed, released and implemented if the normal release process is not appropriate to deal with the problem.

As far as release management is concerned, versions and releases are following the same process (the **release process**). For hot-fixes, where urgency is high and timely repair is required, a special (derived) process will be followed, the **hot-fix process**.

Release Progression

The release process is taken as a basis for description, as the hot-fix process is a derived special case of the release process. The purpose of formal release management is to ensure that any software that is used in production has been tested thoroughly, and has passed all tests to **ensure that the service is of sufficient quality** to support the PPP research data repository adequately.

Following the release cycle means that changes are successively (after passing appropriate testing) moved from one environment to the next in line. Failing to pass tests is prohibitive for passing. In the following scheme, the successive environments in the progression are described.

Environment	Function	Access	Data	Testing	Infrastructure
Development (O')	Development testing	DEV	Synthetic	Auto	Local Dev Infra
Master (O)	Integration testing	DEV	Synthetic	Auto	Local Dev infra
Stable (T)	Functional test	DEV + Users	Synthetic	Auto+ Manual	Local Dev infra
Release Candidate (A)	Acceptance test	Users	(subset of) Production	Manual	Production/ACC
Production (P)	Production	Users	Production	None	Production/PRD
Archive	Roll-back possibility	DEV	None	None	None

Development & Deployment

Each of the four environments from the table above (O, T, A and P) have a corresponding branch in the git repository (https://gitlab.pep.cs.ru.nl/pep/core) and a corresponding set of virtual servers. O and T are deployed to local (development) infrastructure. A and P are deployed to the production infrastructure runned by the management partners. Deployment to this infrastructure is synchronized to well defined and agreed moments. Except for the acceptance tests, all testing and deployment is performed automatically within gitlab when a new commit appears on the branch corrresponding to one of these environments. As for acceptance tests: canary data will be used in Acceptance and Production. Participant with salesforce id 1111...111 is a dummy entry that can be used to verify the correct working. This is used for automatic monitoring and shallow testing (as we cannot write additional values in P however, this is a superficial read test)

Authorizations

Promotions from one environment to another are authorized by DEV (Bernard or Joeri) after verification of successful test results, except for the promotion from RC (A) to Production (P). This promotion is authorized by the user organization (Joost or Jessica).

Procedures

Release procedure

New releases can be released by merging them into Stable (T), if the functional tests pass into Release Candidate (A), and if the acceptance tests pass, into Production (P). In order to explicitly record history in the git repository these merges must never be fast-forward (this is default in gitlab, on the command line one can do “git merge --no-ff”).

Hot-Fix procedure

When a critical error occurs in the production environment the release procedure above may take too long to complete. In this case, the following hot-fix procedure applies:

1. **Disable** the branch protections in gitlab (Settings → Repository)
2. **Create** a new feature-branch starting at the commit that P is on in which to apply the hotfix, and solve the critical error in this branch;
3. The hotfix feature branch must now be **acceptance** tested. There are two possibilities:
    - If A is at the same commit as P, just merge the hotfix feature branch into A.
    - If A is not at the same commit as P but ahead of it (i.e., a new release candidate is being acceptance tested), A must be put back to the contents of P before the hotfix can be applied to it, as we want only the hotfix to be tested and not also whatever happened to be in A. When on the hotfix branch, this can be done by “git push --force rc”. This forcibly puts the rc branch at the same commit as the hotfix branch.
4. If the acceptance tests pass,
    - **Deploy** the hotfix branch to production by merging A (which now contains the hotfix) into P.
    - Also **merge** the hotfix branch into O and T.
5. **Re-enable** branch protections in gitlab.

BN: About the hotfix git strategies. It should integrate with the GitLab environments functionality so we can use that to debug problems and see what version was running at which point in time. Maybe we should run a quick check in a dummy repos to check if our preferred strategy do not interfere with this.